

B.SC. ENGG. PROJECT

SELLWISE (Predictive Sales & Inventory Analytics)

Developed by

Md. Sohel Rana (ID: 222351083)

Md. Labu Miah (ID: 222351215)

Faysal Islam Fahad (ID: 222351133)

Naushin Sultana Mim (ID: 222351203)

Milhan Jannat Jarin (ID: 222351125)

Supervised by

Humaira Ahmed

Assistant Professor

Department of Computer Science & Engineering

(In partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science & Engineering)



Department of Computer Science & Engineering

Bangladesh University of Business & Technology (BUBT)

Rupnagar, Mirpur-2, Dhaka-1216, Bangladesh

August 30, 2026

Contents

1	INTRODUCTION	1
1.1	Introduction	1
1.2	Project overview	1
1.3	Problem Statement	1
1.4	Motivation	2
1.5	Objectives	2
1.6	Development Tools	2
1.7	Proposed System	3
1.8	Purpose Of The System	3
1.9	Goal	3
2	EXISTING SYSTEM	4
2.1	Introduction	4
2.2	Existing System	4
2.2.1	Shopify Analytics	4
2.2.2	Square for Retail	5
2.2.3	Lightspeed Retail	5
2.3	Supporting Literature	5
2.3.1	Used Diagram	5
2.3.2	Technology Used	6
2.4	Conclusion	6
3	SYSTEM EVALUATION	7
3.1	Introduction	7
3.2	Requirement Analysis	7
3.2.1	Non-Functional Requirements	7
3.2.2	Functional Requirements	7
3.3	Methodology	8
3.3.1	Agile Development Model	8
3.3.2	Gantt Chart	9
3.3.3	Data Flow Diagram	9
3.3.4	ER Diagram	11
3.3.5	Use Case Diagram	12
3.3.6	Activity Diagram	13
3.3.7	Sequence Diagram	14
3.3.8	Class Diagram	15
3.3.9	Database Design	15

4	DESIGN IMPLEMENTATION	16
4.1	User Section	16
4.1.1	Database Connection	16
4.1.2	Login and Sign Up	16
4.1.3	Login and Sign Up Backend	17
4.1.4	Dashboard	18
4.1.5	Dashboard Backend	19
4.1.6	POS Order Screen	19
4.1.7	Order Processing Backend	21
4.1.8	Inventory and Products	21
4.1.9	Inventory and Products Backend	23
4.1.10	Inventory Alerts	23
4.1.11	Inventory Alerts Backend	24
4.1.12	Customers	24
4.1.13	Customer Segmentation (RFM) Backend	26
4.1.14	Machine Learning Forecasting Backend	27
4.1.15	Generated PDF Report	27
4.1.16	PDF Generation Backend	29
4.2	Conclusions	29
5	TESTING	30
5.1	Testing	30
5.2	Validation Criteria	30
5.3	Importance of Testing	30
5.4	Structural Testing	30
5.5	Functional Testing	30
5.6	Acceptance Test Generation	31
5.7	Unit Testing	31
5.8	Integration Testing	31
5.9	Test Data Used	31
5.10	Test Cases	32
5.11	Testing Result	32
5.12	Testing Summary	34
6	USER MANUAL	35
6.1	Introduction	35
6.2	System Requirements	35
6.2.1	Server-Side Requirements (Hosting & Deployment)	35
6.2.2	Client-Side Requirements (End-Users)	36
6.3	User Interface Guide	36
6.3.1	Getting Started and Authentication	36
6.3.2	Dashboard and Analytics	36
6.3.3	Point of Sale (Orders)	36
6.3.4	Inventory and Alerts	37
6.3.5	Customer Management	37
6.4	Conclusion	37

7	CONCLUSION AND FUTURE WORKS	38
7.1	Conclusion	38
7.2	Outcomes	38
7.3	Current Limitations	39
7.4	Future Works	39

List of Figures

3.1	Agile Development Cycle	8
3.2	Project Gantt Chart	9
3.3	Context Level DFD (Level 0)	9
3.4	Level 1 Data Flow Diagram	10
3.5	Entity Relationship Diagram	11
3.6	Use Case Diagram	12
3.7	Activity Diagram for Order Processing	13
3.8	Sequence Diagram for the Dashboard	14
3.9	Class Diagram	15
4.1	Login and Sign Up User Interfaces	17
4.2	Dashboard with KPIs, Sales Charts, and Demand Forecasts	18
4.3	POS Cart and Order Creation Screen	20
4.4	Inventory Product Catalog	22
4.5	Low-Stock and Critical Inventory Alerts	23
4.6	Customer List with RFM Segments	25
4.7	Generated PDF Sales Report	28
5.1	Invalid Password Warning During Login Testing	33
5.2	Out-of-Stock Validation During Order Testing	33
5.3	Product Input Validation Error	34

Chapter 1

INTRODUCTION

1.1 Introduction

SellWise is a web application designed to help retail businesses manage their daily operations more effectively. It combines sales processing, inventory management, and customer tracking into one system. Even today, many small and medium-sized shops use paper registers or basic spreadsheets to keep track of their business. These manual methods are slow, prone to mistakes, and difficult to manage as the business grows. SellWise replaces these older methods with a reliable digital system to keep operations organized.

In addition to handling daily transactions, SellWise uses machine learning to predict how much of a product will be needed in the upcoming weeks. The system is divided into two main parts: an ASP.NET Core web application that manages users and sales, and a separate Python service that runs the demand forecasting. Keeping these two parts separate ensures that the main website remains fast and responsive, even when complex calculations are happening in the background.

1.2 Project overview

This project provides a practical and affordable software solution for small to medium-sized retail shops. It is built with a simple user interface so that shop owners and employees can use it without needing advanced technical skills.

SellWise uses a multi-tenant design, which means that each shop gets its own secure workspace and data from different stores is kept completely separate. By handling the difficult tasks of record-keeping, tracking stock, and predicting future sales, the system allows shop owners to focus on growing their business instead of managing paperwork.

1.3 Problem Statement

Without proper management software, small businesses face several common problems. Depending on manual records or disconnected tools usually leads to:

- **Inventory Issues:** Shops often tie up their money by overstocking items that are not selling well, or they run out of popular items and lose potential sales.
- **Losing Customers:** Because store owners cannot easily track what customers are buying over time, they often fail to notice when a regular customer stops visiting.

- **Scattered Information:** Keeping sales records in one place and inventory in another makes it very difficult to calculate the actual profit or loss of the business.
- **No Future Planning:** Traditional methods only show what has already happened. Owners do not have the tools to predict what products they will need to order for the next month.

1.4 Motivation

Seeing the struggles that small shop owners face every day motivated us to build SellWise. We wanted to create a system that brings advanced analytics to small businesses. Our main motivations are to:

- Give shop owners a clear, single dashboard to monitor their entire business.
- Help businesses plan their inventory accurately by forecasting future demand.
- Track customer buying habits automatically to improve customer service.
- Provide an affordable and efficient alternative to manual record-keeping.
- Prevent data loss and the calculation errors that frequently occur with paper records.

1.5 Objectives

- Build a responsive and easy-to-use web platform for managing sales and stock.
- Implement a machine learning model to forecast product demand for the next 30 days.
- Ensure that database transactions are processed safely, so inventory data is never corrupted during a sale.
- Automatically group customers based on their purchase history using the RFM (Recency, Frequency, Monetary) method.
- Create an automated alert system that warns managers when a product's stock is running low.
- Generate downloadable PDF reports that summarize daily and monthly sales.
- Reduce the amount of manual paperwork required to run a shop.

1.6 Development Tools

SellWise is built using a combination of a relational database, a web framework, and a machine learning service.

- **Technology:**

- SQL Server (running in Docker) for the database.
 - ASP.NET Core 10 MVC for the main web application.
 - FastAPI (Python) for the machine learning API.
 - Meta Prophet for time-series forecasting.
 - Entity Framework Core 10 for database interactions.
- **Languages:**
 - C#
 - Python
 - HTML5, CSS3 (Bootstrap 5), and JavaScript
 - SQL

1.7 Proposed System

The proposed system includes three main types of users: Business Owner, Store Manager, and Employee. Users log into the system with their email and password. A single owner account can manage multiple store locations from one place.

The **Owner** has full control over all settings and reports. The **Manager** can process sales, check inventory levels, and view business analytics. The **Employee** is only allowed to process sales orders and view the product list.

When an order is placed, the system immediately checks the inventory. If the stock is too low, it prevents the sale or shows a warning. At the same time, a background service looks at past sales data to predict future demand, which is displayed on the owner's dashboard.

1.8 Purpose Of The System

Many small shop owners know they need better software, but they cannot afford the expensive systems used by large companies. The purpose of SellWise is to make these advanced tools accessible to small businesses.

By putting the point-of-sale, customer tracking, and inventory management into one application, owners do not have to switch between different programs. The system handles the complex math—like figuring out which customers are the most loyal and predicting what will sell next month—so small businesses have a better chance to compete and grow.

1.9 Goal

The main goal of this project is to create an easy-to-use system that makes running a retail shop much more efficient. Another important goal is to improve how owners make decisions by giving them accurate data and future predictions. Ultimately, we want to provide a reliable online platform so that business owners can check their store's performance from anywhere at any time.

Chapter 2

EXISTING SYSTEM

2.1 Introduction

Using technology is now essential for a business to grow. While many local shops still use manual methods, using proper software increases speed and reduces mistakes. This chapter analyzes existing retail software to understand what features are necessary. It also looks at the tools currently available in the market to identify their limitations, especially why small shops often avoid using them.

2.2 Existing System

We examined several popular retail management systems. The details are discussed below.

2.2.1 Shopify Analytics

Shopify is a well-known e-commerce platform that includes built-in analytics. It helps sellers track their sales, website visitors, and inventory.

Advantages:

- Very easy to use with clear dashboard designs.
- Connects online stores with physical point-of-sale systems.
- Tracks inventory and sends basic low-stock warnings.

Disadvantages:

- The more advanced analytics features require expensive monthly plans.
- It does not have built-in machine learning to predict future product demand.
- Users often have to buy third-party apps to get all the features they need.

2.2.2 Square for Retail

Square provides a popular point-of-sale system for small businesses that includes basic inventory and reporting features.

Advantages:

- The basic point-of-sale and inventory tracking software is free.
- Provides reliable sales reports and history.
- Keeps a directory of customers and what they have bought.

Disadvantages:

- Detailed analytics are only available if you pay for the Square Analytics upgrade.
- It does not predict future sales trends automatically.
- It can be difficult to manage multiple store locations using the free version.

2.2.3 Lightspeed Retail

Lightspeed is a more advanced retail system made for detailed inventory management and data analysis.

Advantages:

- Very detailed inventory tracking, which is great for stores with multiple branches.
- Provides excellent custom reports and sales analytics.
- Includes tools for managing suppliers and creating purchase orders.

Disadvantages:

- The price is generally too high for small startups.
- It lacks machine learning tools for automated demand forecasting.
- The system is complex, meaning it takes a lot of time to set up and train staff.

2.3 Supporting Literature

2.3.1 Used Diagram

Entity Relationship Diagram (ERD)

An Entity-Relationship model shows the logical structure of a database using a diagram. It acts as a blueprint for how data is stored. The main parts are:

- **Entity:** An object or concept (like a User or Product) stored in the database. In a database, this is usually a table.
- **Attribute:** The details or properties that describe the entity.

- **Relationship:** Shown as a diamond shape, this describes how different entities interact with each other (for example, a Customer makes an Order).

Data Flow Diagram (DFD)

A Data Flow Diagram shows how data moves through a system. It shows where the data comes from, where it goes, and where it is stored. The main parts are:

- **Process:** An action that takes input data and changes it into output data.
- **Data Flow:** The path the data takes between processes and storage.
- **Data Store:** A place where data is kept so it can be used later.
- **External Entity:** People or other systems outside the software that send or receive data.

Use Case Diagram

A Use Case Diagram gives a simple overview of what the system does from the user's point of view. It shows how people interact with the software.

- **System:** The boundary that contains all the actions the software can perform.
- **Actors:** The people (like the Owner or Manager) who use the system.

2.3.2 Technology Used

To build SellWise, we used ASP.NET Core for the web application, HTML, CSS, and JavaScript for the user interface, and Python for the machine learning features.

- **ASP.NET Core MVC:** This framework was used to build the main website. It separates the database code (Model), the user interface (View), and the logic (Controller).
- **Entity Framework Core:** This tool allows us to interact with the SQL Server database using C# code instead of writing complex SQL queries.
- **Python FastAPI:** We used this fast web framework to create the machine learning service as a separate API.
- **Meta Prophet:** An open-source forecasting tool used to predict future sales based on past data.
- **Bootstrap 5 & Chart.js:** These tools were used to make the website look good on all devices and to draw the graphs on the dashboard.

2.4 Conclusion

While there are many software options for retail management, most of them are either too basic or too expensive and complicated. SellWise fills this gap by offering advanced features—like predicting sales and grouping loyal customers—in a package that is simple and affordable for small businesses.

Chapter 3

SYSTEM EVALUATION

3.1 Introduction

Retail management software covers many areas, from simply recording sales to complex online reporting. After seeing what was missing in basic software, we built SellWise using modern web tools and machine learning. By combining sales records, inventory warnings, and future predictions into one system, we aim to give shop owners tools that are easy to use and actually help them make better business choices.

3.2 Requirement Analysis

Requirement Analysis is the process of defining exactly what the software needs to do before starting to build it. It makes sure the final product meets the users' needs.

3.2.1 Non-Functional Requirements

Non-functional requirements describe how well the system should perform its tasks:

- **Performance:** The dashboard must load quickly, and saving an order should take only a few seconds.
- **Security:** Passwords must be encrypted, and users can only access pages allowed by their specific role (Owner, Manager, or Employee).
- **Reliability:** If the system crashes while an order is being saved, the database must not be corrupted.
- **Manageability:** The code should be organized so that it is easy to update in the future.
- **Usability:** The website must adapt to different screen sizes, working well on both computers and mobile phones.

3.2.2 Functional Requirements

Functional requirements list the specific features the system must have:

- Users must be able to register, log in, and log out.

- The system must allow users to add, edit, and delete products.
- Users can add items to a cart and complete a sales order.
- The system must have a dashboard showing sales statistics and graphs.
- The dashboard must show predicted future demand for products.
- The system must alert the user when a product is running low on stock.
- The system must save customer details and calculate their loyalty score.
- Users must be able to download sales reports as PDF files.

3.3 Methodology

3.3.1 Agile Development Model

For this project, we used the Agile Development Model. Agile focuses on building the software in small, manageable pieces and making improvements based on testing and feedback.

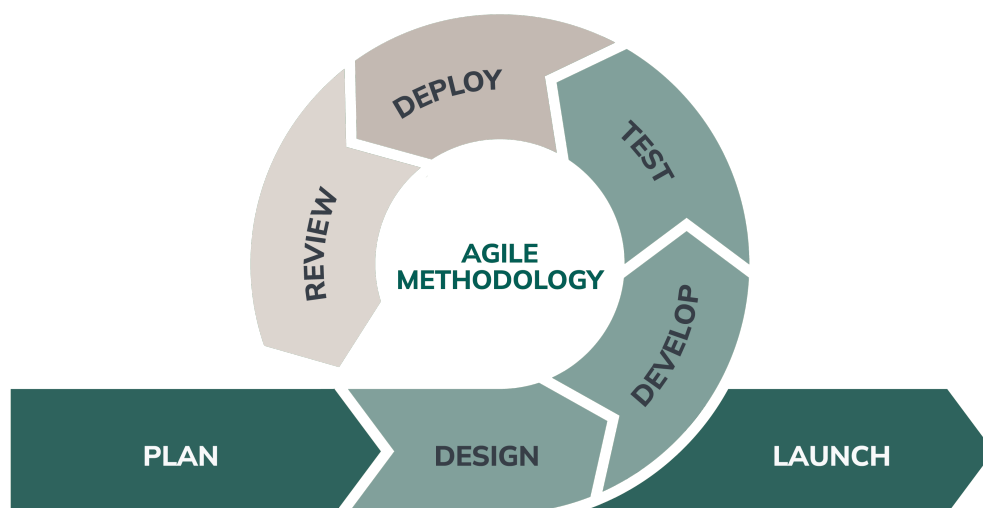


Figure 3.1: Agile Development Cycle

Reasons for choosing the Agile Model:

- It allows us to build and test features one by one, making it easier to fix mistakes early.
- It is flexible, so if we needed to change a requirement during development, we could do so easily.
- It encourages regular communication and testing throughout the project.
- Bugs are found and fixed during the development cycle rather than waiting until the very end.

3.3.2 Gantt Chart

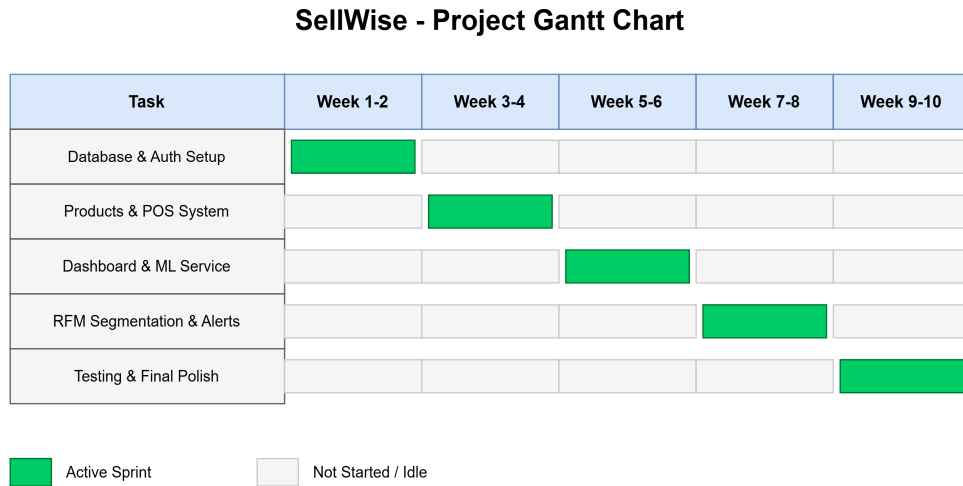


Figure 3.2: Project Gantt Chart

3.3.3 Data Flow Diagram

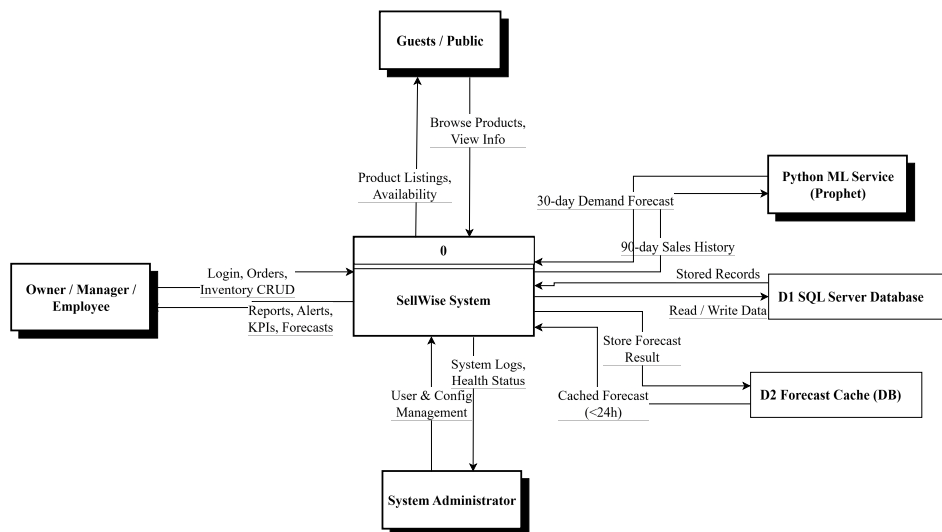


Figure: Context Level DFD of SellWise

Figure 3.3: Context Level DFD (Level 0)

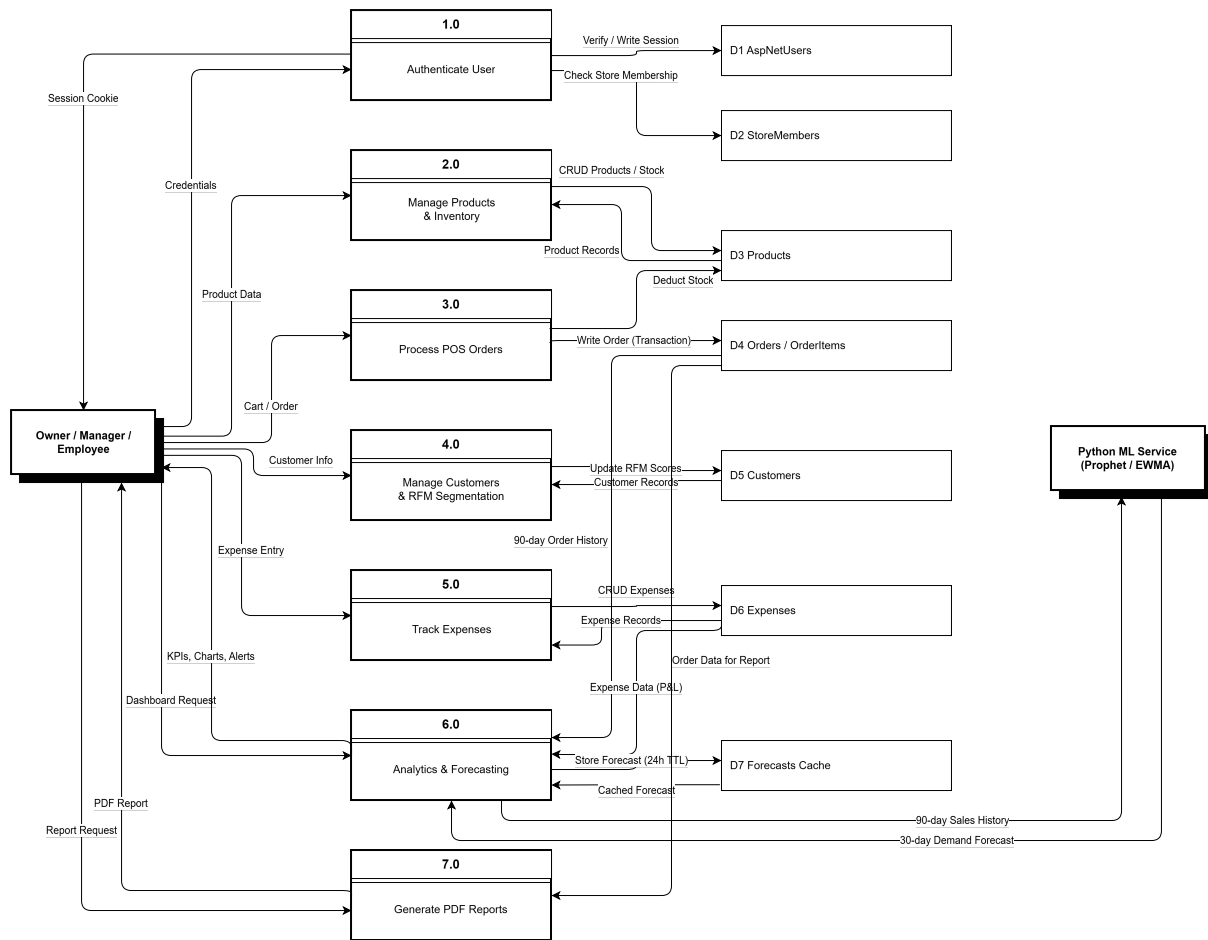


Figure: Level 1 DFD of SellWise

Figure 3.4: Level 1 Data Flow Diagram

3.3.4 ER Diagram

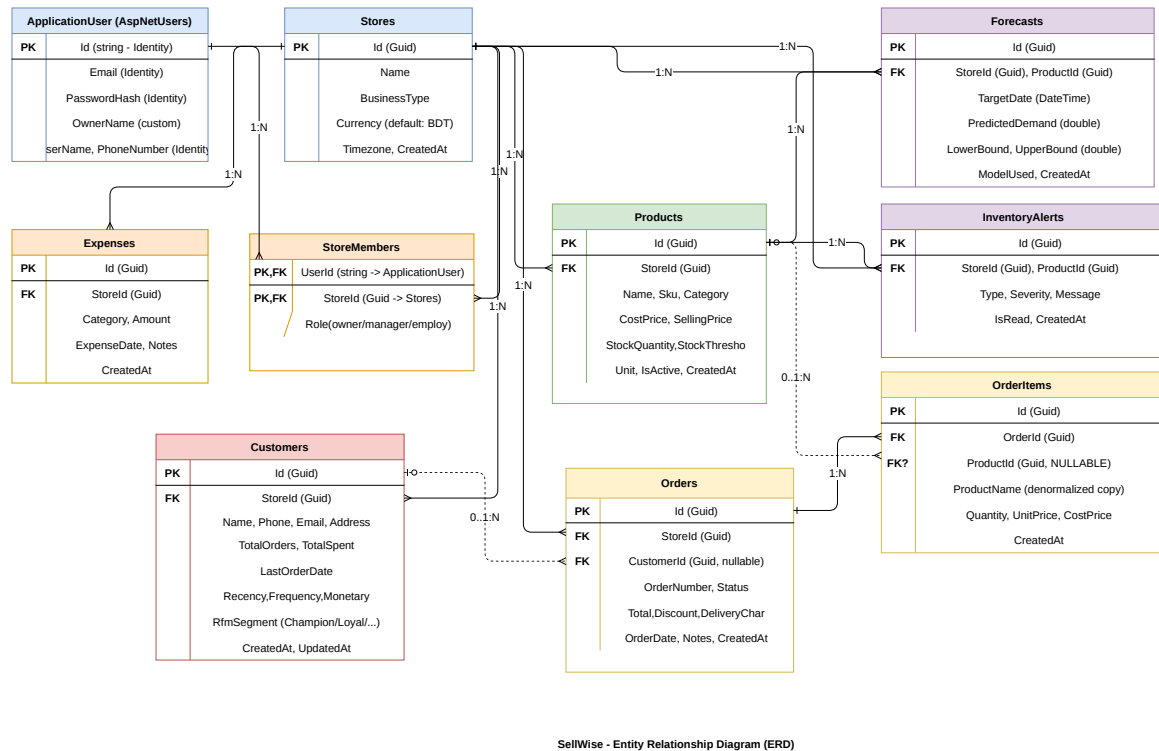


Figure 3.5: Entity Relationship Diagram

Key Tables:

- Users and Stores: User, Store, StoreMember
- Sales Data: Product, Order, OrderItem
- Other Records: Customer, Expense, InventoryAlert, Forecast

3.3.5 Use Case Diagram

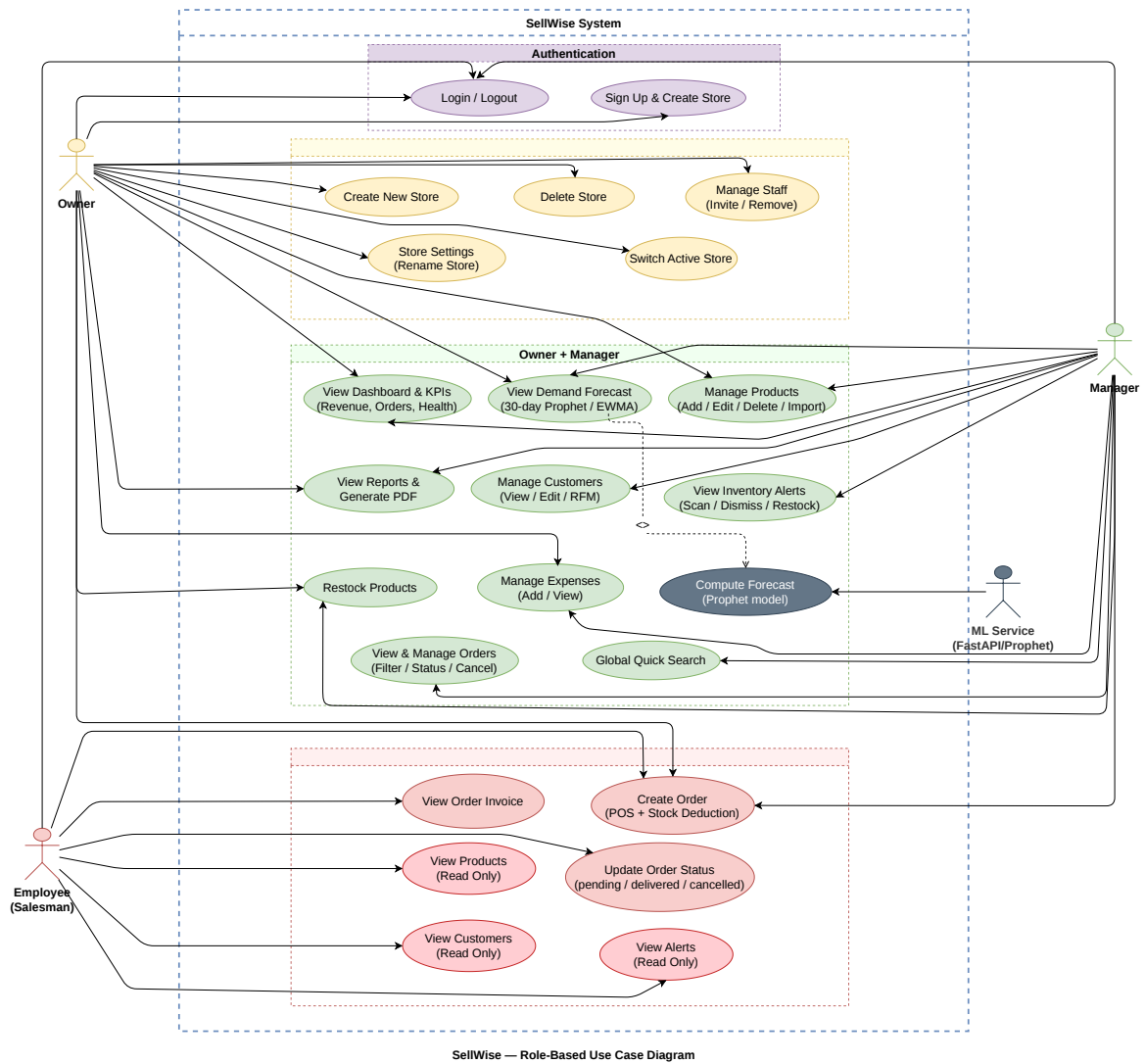


Figure 3.6: Use Case Diagram

- **Owner:** Can manage staff, view the dashboard, switch between stores, and download reports.
- **Manager:** Can process orders, update inventory, and manage customer profiles.
- **Employee:** Can only process sales and look at the product list.

3.3.6 Activity Diagram

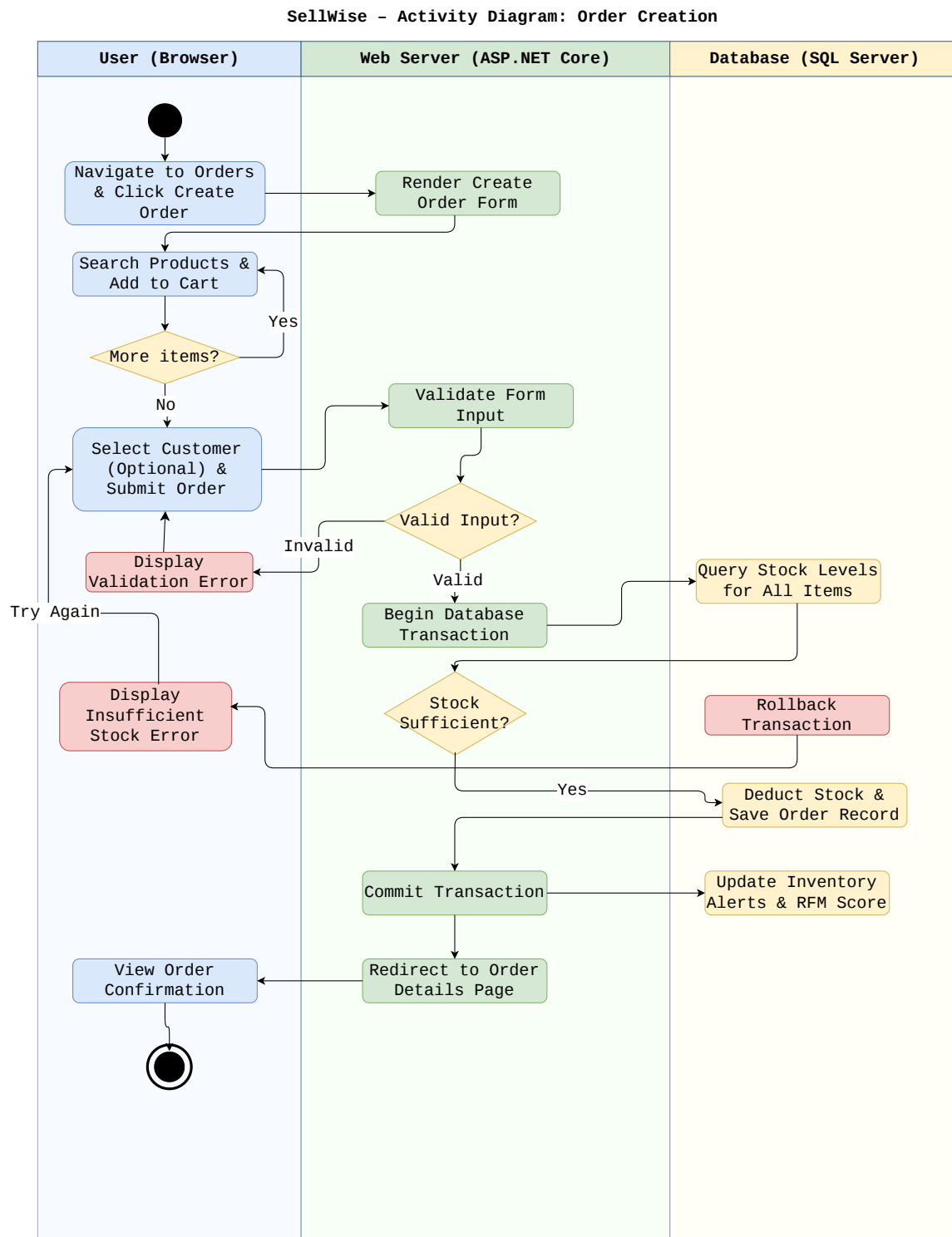


Figure 3.7: Activity Diagram for Order Processing

How an order is processed:

The user clicks "Create Order" → Adds items to the cart → Selects a customer → Submits the form → The system starts a database transaction → It checks if there is enough stock → If yes, it reduces the stock and saves the order → The transaction is completed successfully.

3.3.7 Sequence Diagram

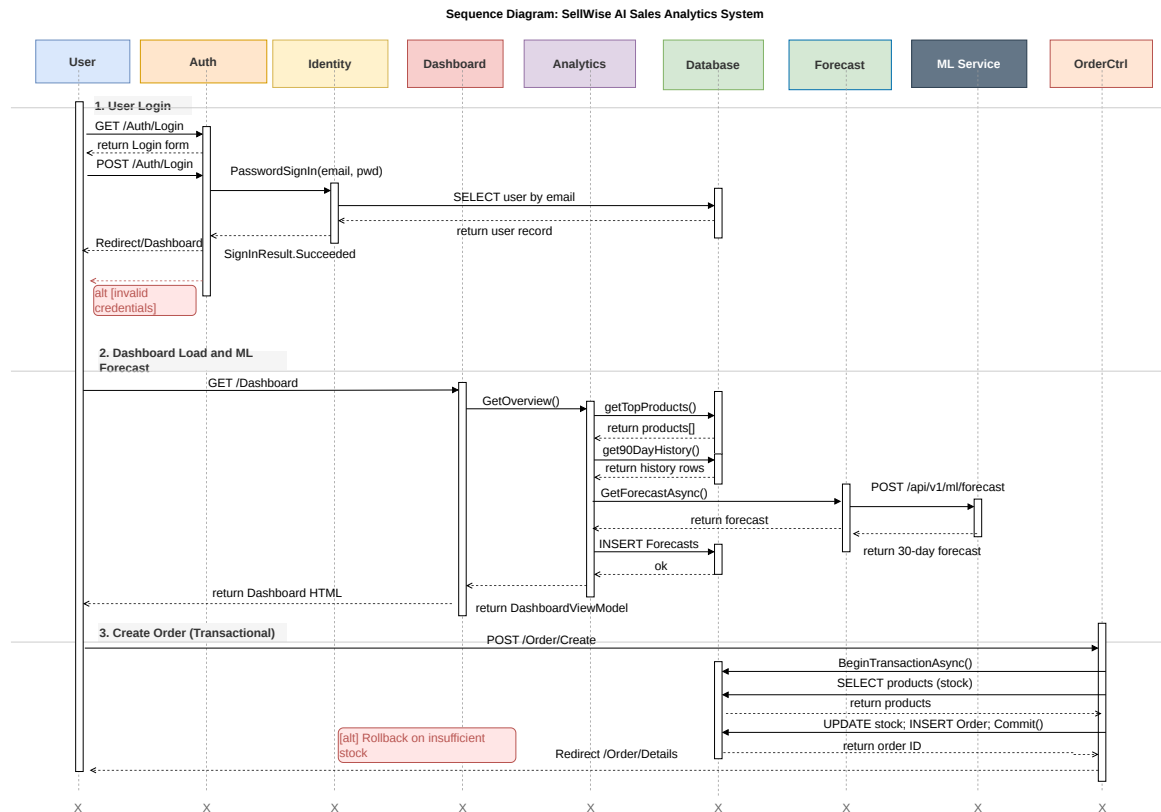


Figure 3.8: Sequence Diagram for the Dashboard

How the dashboard loads data:

The browser requests the dashboard page → AnalyticsService checks the database for past sales → It checks if a recent forecast is saved → If not, it sends a request to the Python ML Service → Python calculates the prediction and sends it back → AnalyticsService saves the new prediction → The dashboard is displayed to the user.

3.3.8 Class Diagram

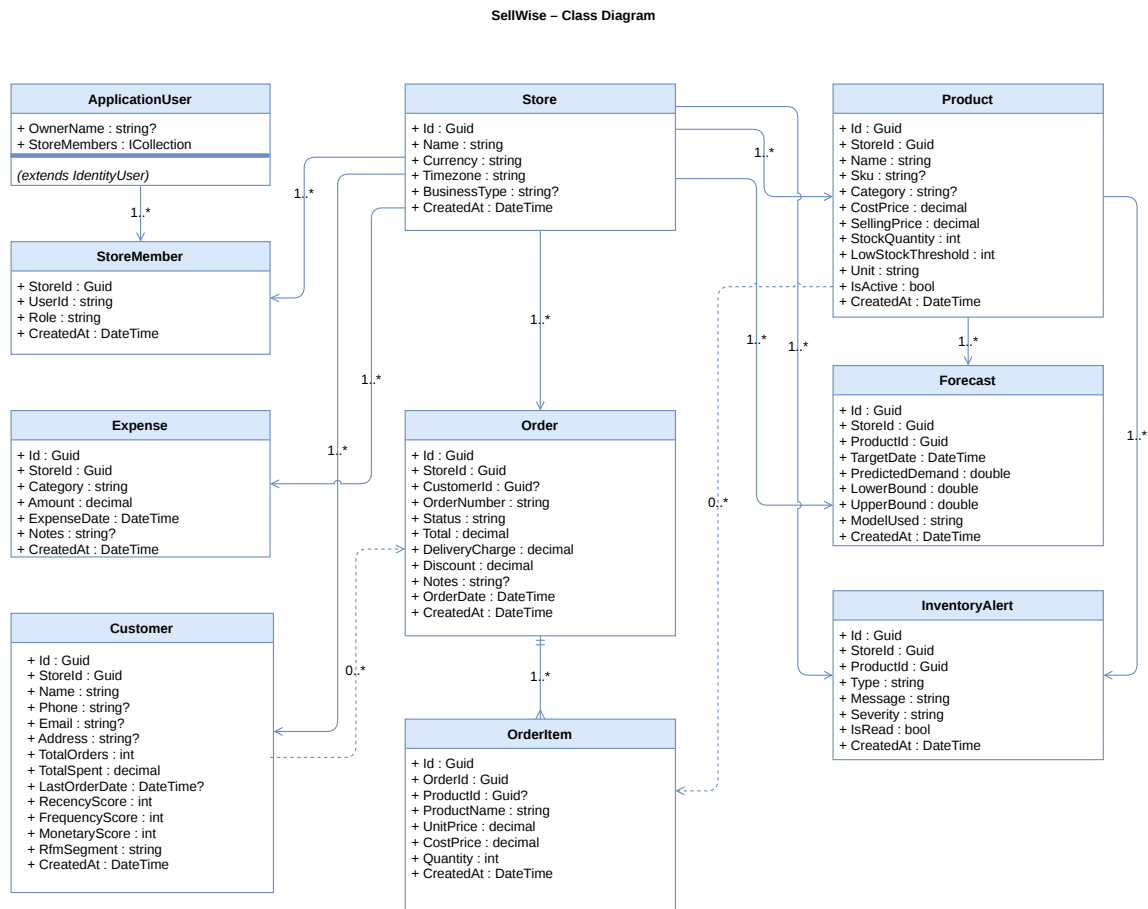


Figure 3.9: Class Diagram

- **BaseController:** Checks which store the user is currently logged into.
- **DashboardController and OrderController:** Use the BaseController to ensure data is saved to the correct store.
- **AnalyticsService:** Does the math for the dashboard statistics.
- **ForecastService:** Communicates with the Python machine learning program.

3.3.9 Database Design

Important tables in our database include:

- **Products:** Keeps track of the product name, price, and how many are in stock.
- **Orders:** Saves the total price and date of a sale.
- **OrderItems:** Connects an Order to specific Products and records the quantity sold.
- **Customers:** Saves customer contact details and their calculated loyalty scores.

Chapter 4

DESIGN IMPLEMENTATION

4.1 User Section

In our system, users are divided into three roles to control what they can do:

- Owner
- Manager
- Employee

Below are the details of how the main features were implemented in the background.

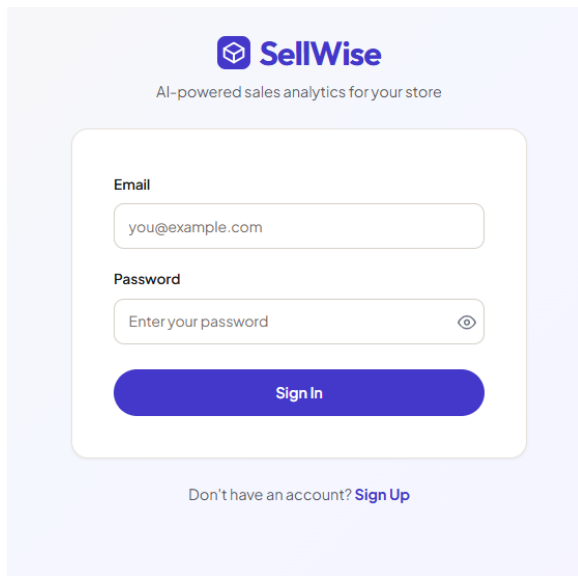
4.1.1 Database Connection

The application connects to the SQL Server database using Entity Framework Core. The connection details are safely stored in the `appsettings.json` file.

```
1 {  
2   "ConnectionStrings": {  
3     "DefaultConnection": "Server=localhost,1433;Database=SellWise;  
4     User Id=sa;Password=YourPass123!;TrustServerCertificate=True;"  
5   }  
6 }
```

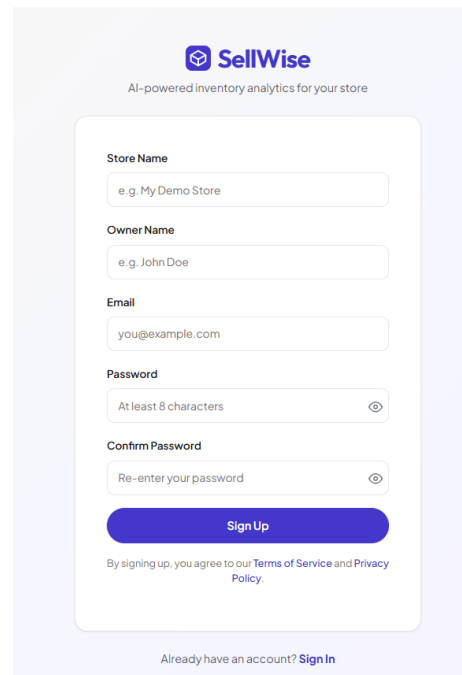
4.1.2 Login and Sign Up

The authentication interface provides separate screens for existing users to log in and new users to create an account.



The login screen features the SellWise logo and tagline 'AI-powered sales analytics for your store'. It contains a form with fields for 'Email' (with placeholder 'you@example.com') and 'Password' (with placeholder 'Enter your password' and a toggle icon). A blue 'Sign In' button is at the bottom. Below the button is a link: 'Don't have an account? [Sign Up](#)'.

Login screen



The sign up screen features the SellWise logo and tagline 'AI-powered inventory analytics for your store'. It contains a form with fields for 'Store Name' (placeholder 'e.g. My Demo Store'), 'Owner Name' (placeholder 'e.g. John Doe'), 'Email' (placeholder 'you@example.com'), 'Password' (placeholder 'At least 8 characters' with a toggle icon), and 'Confirm Password' (placeholder 'Re-enter your password' with a toggle icon). A blue 'Sign Up' button is at the bottom. Below the button is a link: 'By signing up, you agree to our [Terms of Service and Privacy Policy](#)'. At the very bottom is a link: 'Already have an account? [Sign In](#)'.

Sign Up screen

Figure 4.1: Login and Sign Up User Interfaces

4.1.3 Login and Sign Up Backend

When a user logs in, ASP.NET Core Identity checks their password. If it is correct, the system saves their Store ID in a secure cookie. This makes sure that the user only sees data belonging to their specific store.

```

1 [HttpPost]
2 public async Task<IActionResult> Login(
3     LoginViewModel model)
4 {
5     if (!ModelState.IsValid)
6         return View(model);
7
8     var result = await _signInManager
9         .PasswordSignInAsync(
10             model.Email!,
11             model.Password!,
12             isPersistent: true,
13             lockoutOnFailure: false);
14
15     if (result.Succeeded)
16     {
17         var user = await _userManager
18             .FindByEmailAsync(model.Email!);
19
20         // Redirect to dashboard
21     }
22
23     return View(model);
24 }

```

4.1.4 Dashboard

The dashboard presents the main sales KPIs, interactive charts, and Prophet-based demand forecasts in one view.

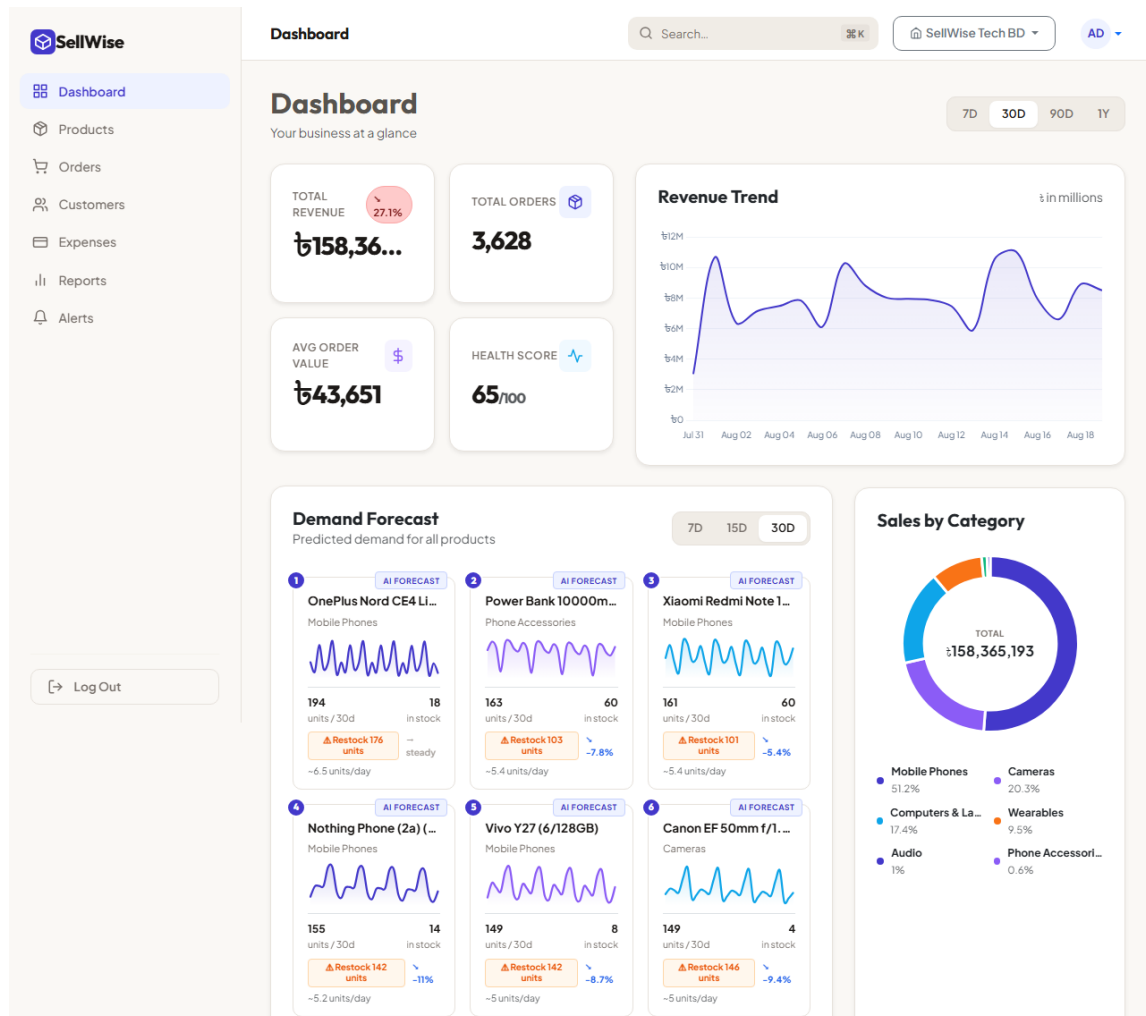


Figure 4.2: Dashboard with KPIs, Sales Charts, and Demand Forecasts

4.1.5 Dashboard Backend

The analytics service calculates today's revenue, order counts, and pending orders for the active store to display on the dashboard.

```
1 public async Task<(
2     decimal TodayRevenue,
3     int TodayOrdersCount,
4     int PendingOrdersCount)> GetDashboardKpisAsync(
5     Guid storeId)
6 {
7     var today = DateTime.UtcNow.Date;
8
9     var kpis = await _db.Orders
10         .Where(o => o.StoreId == storeId)
11         .GroupBy(o => 1)
12         .Select(g => new
13         {
14             TodayRevenue = g
15                 .Where(o => o.OrderDate >= today &&
16                     o.Status != "cancelled")
17                 .Sum(o => o.Total),
18             TodayOrdersCount = g.Count(
19                 o => o.OrderDate >= today),
20             PendingOrdersCount = g.Count(
21                 o => o.Status == "pending" ||
22                 o.Status == "processing")
23         })
24         .FirstOrDefaultAsync();
25
26     return (
27         kpis?.TodayRevenue ?? 0m,
28         kpis?.TodayOrdersCount ?? 0,
29         kpis?.PendingOrdersCount ?? 0);
30 }
```

4.1.6 POS Order Screen

The point-of-sale screen allows users to select products, manage the cart, choose a customer, and complete an order.

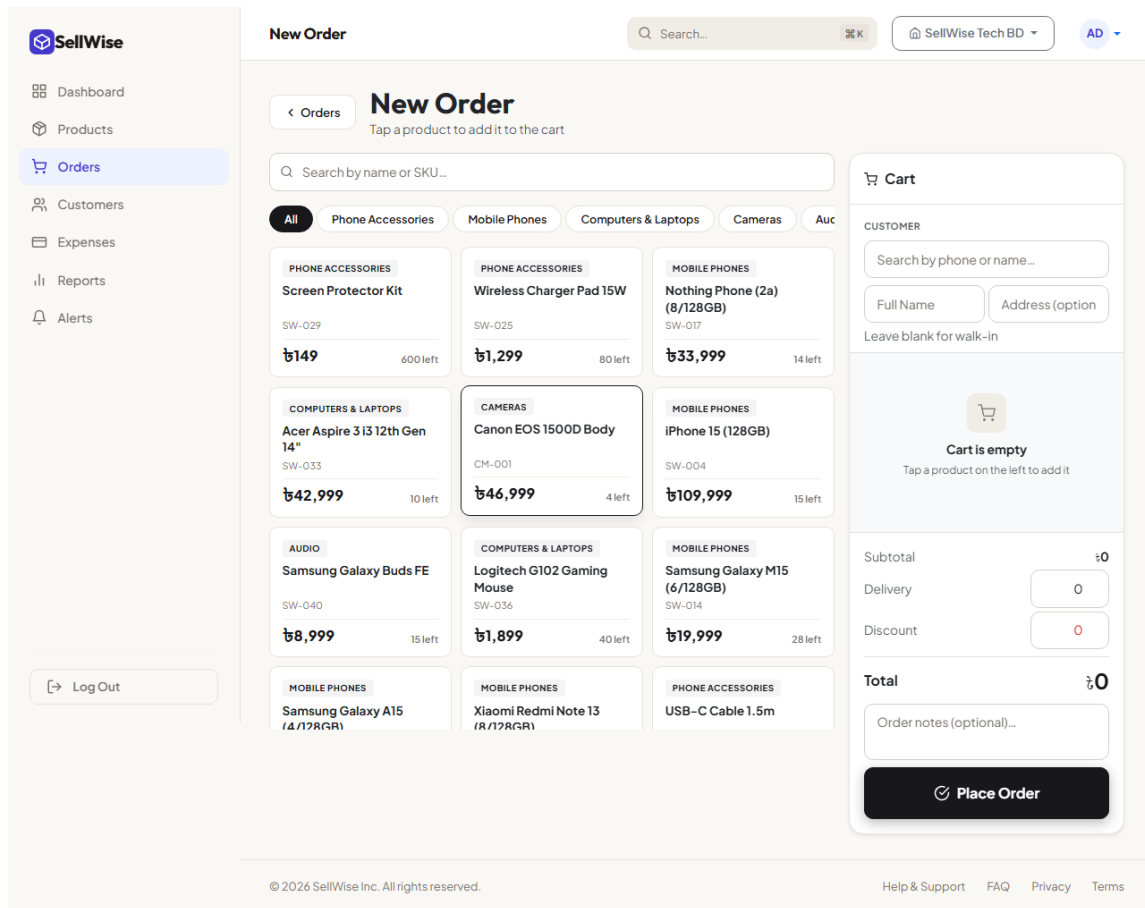


Figure 4.3: POS Cart and Order Creation Screen

4.1.7 Order Processing Backend

To prevent errors when selling items, the system uses database transactions. This means that deducting the stock from the inventory and saving the payment record happen together.

```
1 public async Task<string?> CreateOrderAsync(Guid storeId, OrderFormViewModel model
  )
2 {
3     using var transaction = await _db.Database.BeginTransactionAsync();
4     try
5     {
6         Guid? finalCustomerId = model.CustomerId;
7         // ... (customer setup omitted for brevity) ...
8
9         var order = new Order
10        {
11            StoreId = storeId,
12            CustomerId = finalCustomerId,
13            Status = "pending",
14            OrderNumber = $"ORD-{DateTime.UtcNow:yyyyMMdd}-{Guid.NewGuid().
ToString()[..6]}"
15        };
16
17        decimal subtotal = 0;
18        var validItems = model.Items.Where(i => i.ProductId != Guid.Empty && i.
Quantity > 0).ToList();
19
20        foreach (var item in validItems)
21        {
22            var product = await _db.Products.FindAsync(item.ProductId);
23            if (product == null || product.StockQuantity < item.Quantity)
24            {
25                await transaction.RollbackAsync();
26                return $"Insufficient stock for {product?.Name ?? "product"}.";
27            }
28            product.StockQuantity -= item.Quantity; // Deduct stock safely
29            // ...
30        }
31        await _db.SaveChangesAsync();
32        await transaction.CommitAsync();
33        return null;
34    }
35    catch (Exception ex)
36    {
37        await transaction.RollbackAsync();
38        return "Failed to place order.";
39    }
40 }
```

4.1.8 Inventory and Products

The product catalog displays the available products and provides controls for maintaining prices, quantities, and other inventory information.

- Dashboard
- Products**
- Orders
- Customers
- Expenses
- Reports
- Alerts

Log Out

Products

Search...

SellWise Tech BD

AD

Import

Add Product

Search products...

NAME	SKU	PRICE	STOCK	
Xiaomi Smart Band 8	SW-043	₹3,999	80 pcs	
Samsung Galaxy Watch 6 (40mm)	SW-042	₹29,999	18 pcs	
Apple Watch Series 9 (45mm)	SW-041	₹54,999	150 pcs	
Amazonbasics Tripod (60 -inch)	CM-009	₹3,999	15 pcs	
Amazfit GTR 4 Smartwatch	CM-010	₹18,999	12 pcs	
Camera Bag (Large)	CM-008	₹2,499	25 pcs	
Canon EOS 1500D Body	CM-001	₹46,999	4 pcs	
Nikon D3500 Body	CM-002	₹43,999	3 pcs	
GoPro Hero 12 Black	CM-003	₹42,999	5 pcs	
DJI Mini 4K Drone	CM-004	₹39,999	3 pcs	
SanDisk Ultra 128GB SD Card	CM-005	₹1,899	100 pcs	
Manfrotto PIXI Mini Tripod	CM-006	₹3,299	20 pcs	
Canon EF 50mm f/1.8 STM Lens	CM-007	₹24,999	4 pcs	
Samsung Galaxy Buds FE	SW-040	₹8,999	15 pcs	
Edifier X200 TWS Earbuds	SW-039	₹1,499	50 pcs	
Baseus 20000mAh Power Bank	SW-038	₹2,999	30 pcs	
Havit HV-H206U Gaming Headset	SW-037	₹2,499	25 pcs	
Logitech G102 Gaming Mouse	SW-036	₹1,899	40 pcs	
Dell Inspiron 15 i5 13th Gen	SW-035	₹67,999	5 pcs	
ASUS VivoBook 15 i5 12th Gen	SW-034	₹59,999	7 pcs	

Showing 20 of 53 results

Previous

Next

© 2026 SellWise Inc. All rights reserved.

[Help & Support](#)
[FAQ](#)
[Privacy](#)
[Terms](#)

Figure 4.4: Inventory Product Catalog

4.1.9 Inventory and Products Backend

When creating a new product, the system automatically assigns it to the current user's store and validates all input constraints before saving to the database.

```
1 public async Task CreateProductAsync(  
2     Guid storeId,  
3     ProductFormViewModel model)  
4 {  
5     var product = new Product  
6     {  
7         StoreId = storeId,  
8         Name = model.Name!,  
9         Sku = model.Sku,  
10        Category = model.Category!,  
11        Price = model.Price,  
12        StockQuantity = model.StockQuantity,  
13        IsActive = true  
14    };  
15  
16    _db.Products.Add(product);  
17    await _db.SaveChangesAsync();  
18 }
```

4.1.10 Inventory Alerts

Low-stock and critical warning badges make products requiring immediate attention easy to identify.

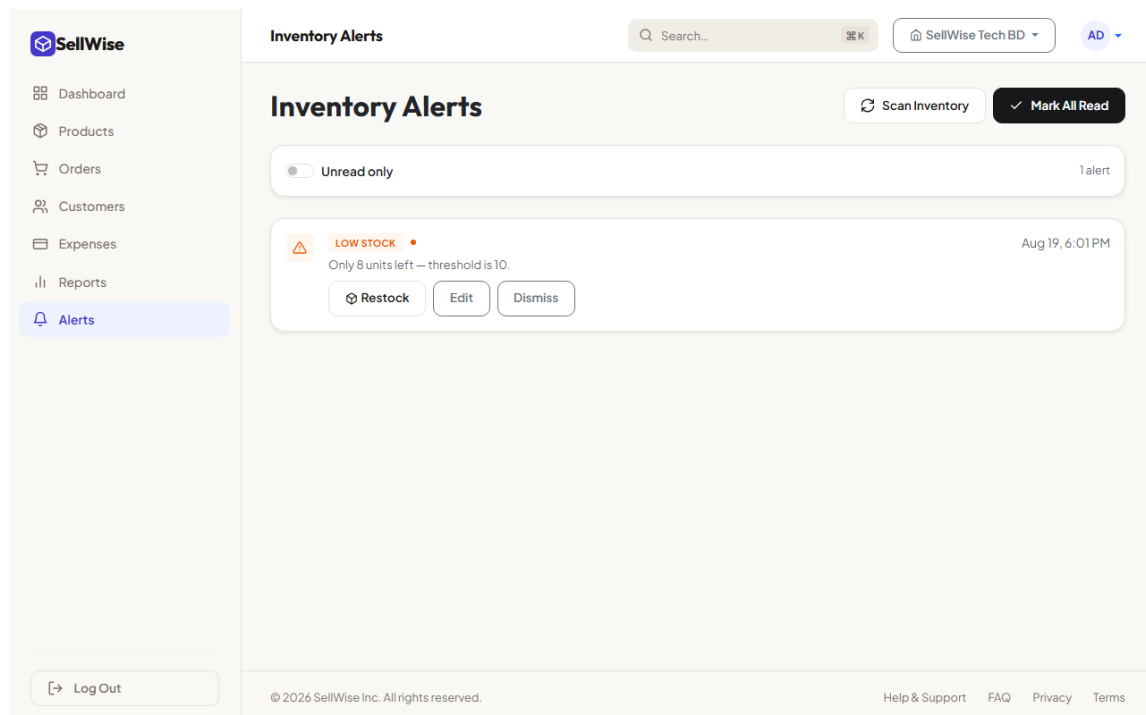


Figure 4.5: Low-Stock and Critical Inventory Alerts

4.1.11 Inventory Alerts Backend

A background task constantly checks the quantity of every product. If a product's stock drops below a set minimum limit, the system automatically creates a warning alert.

```
1 public async Task ScanAndGenerateAlertsAsync(Guid storeId)
2 {
3     // Remove alerts for products that are healthy
4     var healthyOrInactiveProductIds = await _db.Products
5         .Where(p => p.StoreId == storeId && (!p.IsActive || p.StockQuantity > p.
6         LowStockThreshold))
7         .Select(p => p.Id).ToListAsync();
8
9     var alertsToRemove = await _db.Alerts
10        .Where(a => a.StoreId == storeId && (healthyOrInactiveProductIds.Contains(
11        a.ProductId) || a.Type.Contains("Resolved")))
12        .ToListAsync();
13
14    if (alertsToRemove.Any()) _db.Alerts.RemoveRange(alertsToRemove);
15
16    // Generate alerts for low stock products
17    var lowStockProducts = await _db.Products
18        .Where(p => p.StoreId == storeId && p.IsActive && p.StockQuantity <= p.
19        LowStockThreshold)
20        .ToListAsync();
21
22    // ... insert alerts logic ...
23    await _db.SaveChangesAsync();
24 }
```

4.1.12 Customers

The customer list displays customer information together with automatically calculated RFM segments, such as Champion and At Risk.

Customers

Q Search...

SellWise Tech BD

AD

Customers

Update RFM

Q Search by name, phone or email...

All Segments

Newest First

NAME	CONTACT	ORDERS	SEGMENT (RFM)	TOTAL SPENT	ACTIONS
Tasnim Khatun	01763564740 tasnim.khatun11@yahoo.com	82	Loyal R:5 F:3 M:3	৳3,557,621	
Shibli Bhuiyan	01722843680 shibli.bhuiyan67@hotmail.com	93	Loyal R:5 F:4 M:3	৳3,815,357	
Firoz Parvin	01754794150 froz.parvin22@hotmail.com	89	Loyal R:5 F:3 M:3	৳3,524,197	
Limon Sinha	01758950564 limon.sinha59@yahoo.com	84	Loyal R:5 F:3 M:4	৳3,933,003	
Rasel Begum	01743652625 rasel.begum5@yahoo.com	80	New Customer R:5 F:2 M:2	৳3,166,698	
Babul Hasan	01782262413 babul.hasan58@hotmail.com	84	Loyal R:5 F:3 M:3	৳3,780,833	

Figure 4.6: Customer List with RFM Segments

4.1.13 Customer Segmentation (RFM) Backend

SellWise automatically calculates a score for each customer using the RFM model: Recency, Frequency, and Monetary value. Based on this score, customers are grouped.

```
1 public async Task RecalculateAllAsync(Guid storeId)
2 {
3     var customers = await _db.Customers.Where(c => c.StoreId == storeId).
        ToListAsync();
4     if (customers.Count == 0) return;
5
6     var now = DateTime.UtcNow;
7     var cutoffDate = now.AddDays(-365);
8
9     var customerOrders = await _db.Orders
10         .Where(o => o.StoreId == storeId && o.CustomerId.HasValue && o.OrderDate
        >= cutoffDate)
11         .GroupBy(o => o.CustomerId!.Value)
12         .Select(g => new
13         {
14             CustomerId = g.Key,
15             LastOrderDate = g.Max(o => o.OrderDate),
16             Frequency = g.Count(),
17             Monetary = g.Sum(o => o.Total)
18         }).ToListAsync();
19
20     // Calculate quartiles and assign segments (e.g. "Champion", "At Risk")
21     foreach (var customer in customers)
22     {
23         // Segment assignment logic based on Recency, Frequency, Monetary
quartiles...
24         customer.RfmSegment = DetermineSegment(rScore, fScore, mScore);
25     }
26     await _db.SaveChangesAsync();
27 }
```

4.1.14 Machine Learning Forecasting Backend

We created a separate Python program (microservice) to handle predictions. It uses a library called Prophet to look at past sales and predict what will sell in the next 30 days.

```
1 @router.post("/forecast", response_model=ForecastResponse)
2 def create_forecast(request: ForecastRequest):
3     try:
4         # Generate forecast using the prophet service
5         predictions = generate_forecast(
6             request.history,
7             request.periods
8         )
9         return ForecastResponse(
10             store_id=request.store_id,
11             product_id=request.product_id,
12             forecast=predictions
13         )
14     except Exception as e:
15         logger.exception("Forecast generation failed")
16         raise HTTPException(
17             status_code=500,
18             detail="Forecast generation failed."
19         )
```

4.1.15 Generated PDF Report

SellWise can export a formatted PDF sales report so that users can save, print, or share business summaries.

SellWise Sales Report

Period: Last 30 Days (Jul 31, 2026 - Aug 30, 2026)

Generated:
8/30/2026

Total Orders: 3623

Total Revenue: ₳158,106,903

Date	Customer Name	Status	Total
Aug 20, 2026 03:57	Marium Khatun	completed	₳53,998
Aug 20, 2026 03:52	Sabrina Islam	completed	₳39,998
Aug 20, 2026 03:47	Shapna Rahman	completed	₳9,897
Aug 20, 2026 03:46	Firoz Sheikh	completed	₳56,997
Aug 20, 2026 03:46	Shibli Khan	completed	₳32,997
Aug 20, 2026 03:41	Nasrin Uddin	completed	₳9,897
Aug 20, 2026 03:41	Shibli Chowdhury	pending	₳799
Aug 20, 2026 03:38	Sumon Majumder	completed	₳37,998
Aug 20, 2026 03:28	Dipu Sarkar	completed	₳1,299
Aug 20, 2026 03:26	Khadija Ahmed	pending	₳43,999
Aug 20, 2026 03:22	Limon Sinha	completed	₳5,697
Aug 20, 2026 03:17	Emon Sinha	completed	₳5,998
Aug 20, 2026 03:15	Imran Chowdhury	completed	₳56,997
Aug 20, 2026 03:14	Khadija Mollah	completed	₳447
Aug 20, 2026 03:10	Nira Bhuiyan	completed	₳698
Aug 20, 2026 03:10	Shibli Khan	completed	₳21,998
Aug 20, 2026 03:07	Poly Mollah	completed	₳39,998
Aug 20, 2026 02:57	Dipu Sarkar	pending	₳149
Aug 20, 2026 02:55	Tuhin Noor	completed	₳24,999
Aug 20, 2026 02:54	Shohel Hossain	completed	₳89,997
Aug 20, 2026 02:54	Sakib Islam	completed	₳4,497
Aug 20, 2026 02:53	Ruma Talukder	pending	₳4,497

Page 1 of 3

Figure 4.7: Generated PDF Sales Report

4.1.16 PDF Generation Backend

The system generates PDF reports dynamically using the QuestPDF library, providing a clean and easily downloadable summary of sales and orders.

```
1 [Authorize]
2 public class ReportController : BaseController
3 {
4     public ReportController(AppDbContext db)
5         : base(db)
6     {
7         QuestPDF.Settings.License = LicenseType.Community;
8     }
9
10    public async Task<IActionResult> Index()
11    {
12        var storeId = GetCurrentStoreId();
13
14        if (storeId == System.Guid.Empty)
15        {
16            return RedirectToAction("Login", "Auth");
17        }
18
19        var totalOrders = await Db.Orders
20            .Where(o => o.StoreId == storeId)
21            .CountAsync();
22
23        var totalRevenue = await Db.Orders
24            .Where(o => o.StoreId == storeId)
25            .SumAsync(o => o.Total);
26
27        // Render PDF view or return data
28        return View();
29    }
30 }
```

4.2 Conclusions

A few important design choices helped make the system stable:

1. Writing the business calculations in separate "Service" files kept the main web code clean and easy to read.
2. Using database transactions for orders completely prevented issues where stock numbers did not match the sales records.
3. Checking the Store ID for every database request ensured that data from one shop never mixed with another.
4. Adding a backup calculation method means the dashboard still works even if the Python prediction service is turned off.

Chapter 5

TESTING

5.1 Testing

Testing is the process of running the software with test data to check if it works exactly as it is supposed to. During testing, we try to find errors so they can be fixed before the software is given to the final users.

5.2 Validation Criteria

The system was checked against the following rules:

- User inputs must be checked for mistakes before being saved to the database.
- The design and colors of the website must look consistent on every page.
- If the system crashes during an update, the database must not be corrupted.

5.3 Importance of Testing

Testing is very important because it ensures the system is reliable. Finding and fixing bugs during the testing phase prevents the software from crashing when the shop owner is using it to run their business.

5.4 Structural Testing

Structural testing checks the internal code of the software. We reviewed the code to ensure that important database actions are placed inside `try-catch` blocks. This prevents the website from crashing completely if an error happens.

5.5 Functional Testing

Functional testing involves testing the features from the user's perspective. We manually tested logging in, adding products to the cart, processing orders, and checking if the dashboard numbers correctly updated after a sale.

5.6 Acceptance Test Generation

To see how the system handles a lot of data, we wrote a script to generate test data. We created two fake stores, added hundreds of customers, and generated 180 days of past orders. This helped us see how the system would perform in a real-world scenario.

5.7 Unit Testing

Unit testing focuses on checking small, individual parts of the code. For example, we tested the math behind the customer loyalty (RFM) calculation and the code that reads the prediction from the Python service to ensure they gave the correct results.

5.8 Integration Testing

Integration testing checks how different parts of the system work together. We specifically tested the connection between the main C# web application and the Python machine learning service to make sure they communicated properly.

5.9 Test Data Used

- **Live Test Data:** We manually typed in real examples of products and sales to test the website forms.
- **Artificial Test Data:** We used automated scripts to enter thousands of fake orders to test if the database slowed down.

5.10 Test Cases

Module	Test Case Scenarios	Tested	Working
Authentication	1. Check login with correct password 2. Check account locks after 5 wrong attempts 3. Check if user is restricted to their own store	Yes	Yes
Dashboard	1. Verify the sales statistics are correct 2. Check if the graphs show the right data	Yes	Yes
POS Orders	1. Check adding and removing items in the cart 2. Verify an order cannot be placed without enough stock 3. Verify the transaction cancels safely on error	Yes	Yes
Products	1. Test adding, editing, and deleting a product 2. Check if low stock alerts appear correctly	Yes	Yes
Customers	1. Verify the loyalty score calculation is accurate 2. Verify customers are assigned the correct group	Yes	Yes

Table 5.1: Test Cases and Results

5.11 Testing Result

The login system worked perfectly, showing warnings for wrong passwords and locking the account after too many tries. The order processing system also worked safely, correctly deducting stock only when the sale was finalized.

The following screenshots demonstrate validation and error handling during functional testing.

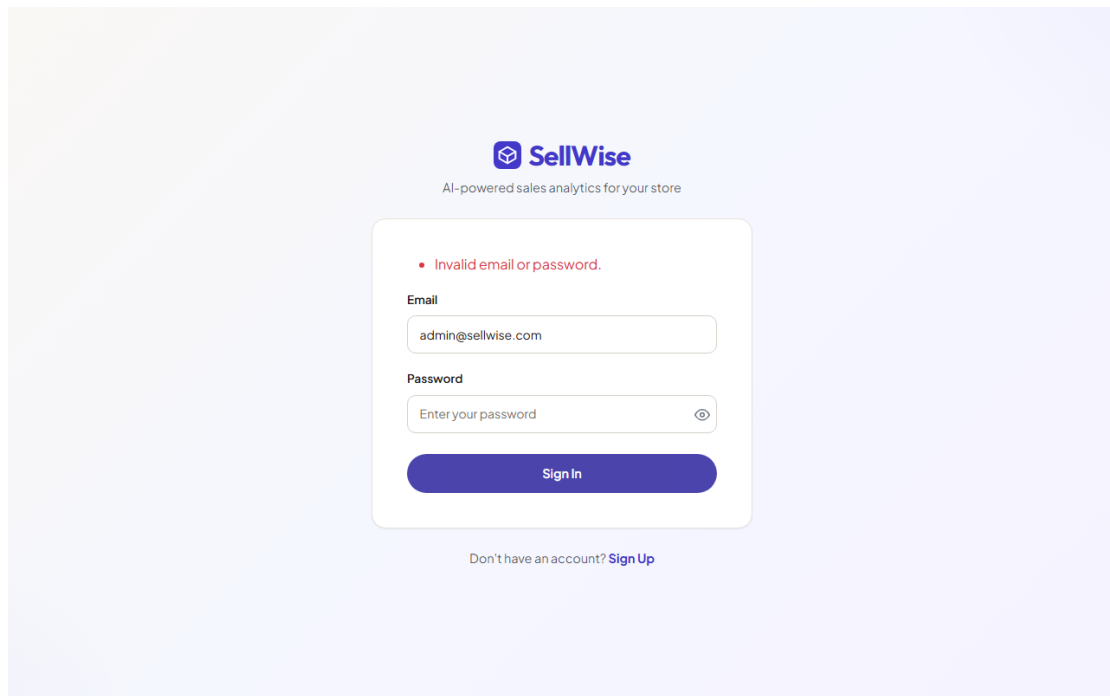


Figure 5.1: Invalid Password Warning During Login Testing

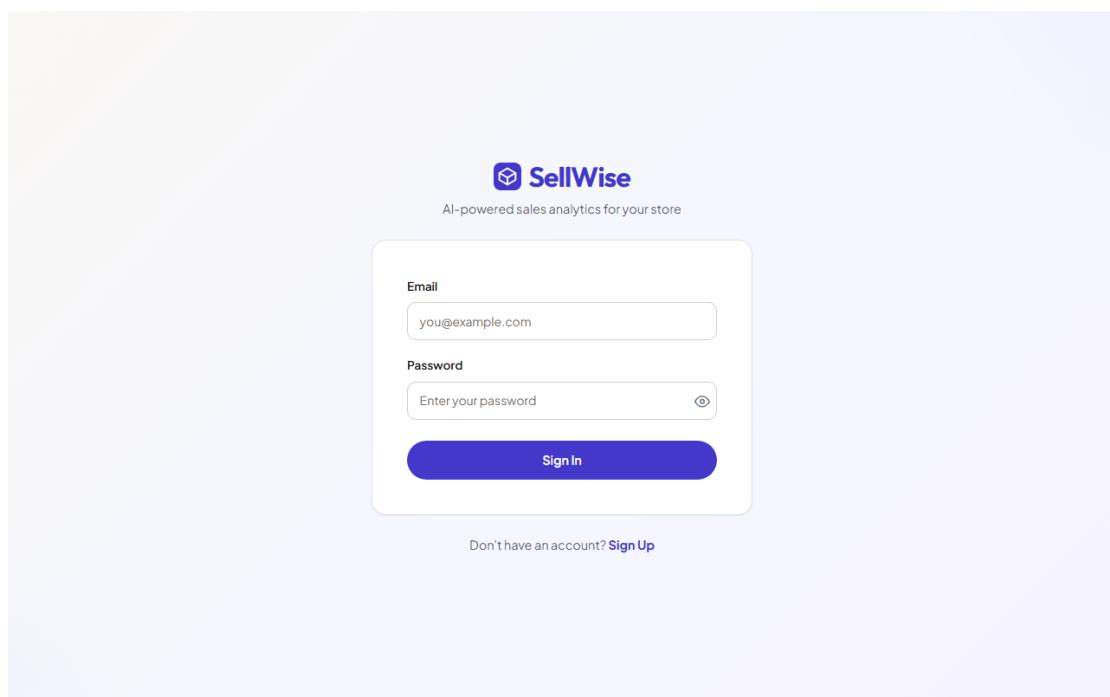


Figure 5.2: Out-of-Stock Validation During Order Testing

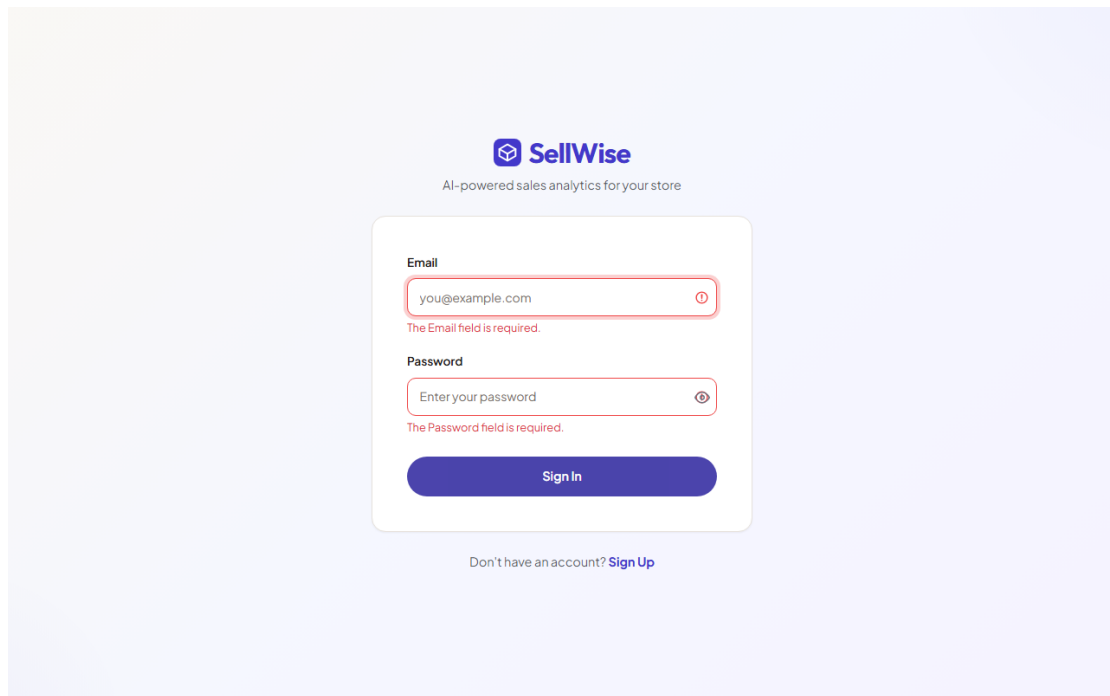


Figure 5.3: Product Input Validation Error

5.12 Testing Summary

We tested all the pages available to the Owner, Manager, and Employee. We checked the login process, order creation, and report generation. After finding and fixing a few minor display issues, the project passed all tests and is working smoothly.

Chapter 6

USER MANUAL

6.1 Introduction

The SellWise User Manual is designed to help shop owners, managers, and employees quickly learn how to use the system. SellWise provides a comprehensive set of tools to manage a retail shop by combining point-of-sale, inventory tracking, and sales forecasting into one accessible platform. Because it is a web-based application, it offers the flexibility to monitor your sales and inventory from any device on your network.

This chapter covers the system requirements necessary to run the application and provides a step-by-step guide to using the core features of the user interface.

6.2 System Requirements

Because SellWise is a web application, the system requirements are divided into two categories: what the server needs to host the application, and what the end-user needs to access it.

6.2.1 Server-Side Requirements (Hosting & Deployment)

To host the SellWise application, run the machine learning forecasts, and manage the database, the server environment must meet the following specifications:

- **Operating System:** Windows 10/11, macOS, or a modern Linux distribution (e.g., Ubuntu).
- **Hardware:** Minimum Dual-Core processor with 8GB of RAM (required to smoothly run Docker and the Python environment simultaneously). At least 50GB of free storage.
- **Database Engine:** Docker Engine (or Docker Desktop) is required to run the Microsoft SQL Server 2022 container.
- **Web Framework:** .NET 10.0 SDK/Runtime to host the main ASP.NET Core MVC application.
- **Machine Learning Environment:** Python 3.11 or higher, configured with the FastAPI framework and the Meta Prophet library for sales forecasting.

6.2.2 Client-Side Requirements (End-Users)

For shop owners, managers, and employees, no installation of the software is required. They only need the following to use the system:

- **Hardware:** Any basic PC, laptop, or tablet.
- **Operating System:** Any OS (Windows, macOS, Linux, Android, iOS).
- **Web Browser:** A modern, up-to-date web browser such as Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge.
- **Network:** A stable internet or local area network (LAN) connection to communicate with the SellWise web server.

6.3 User Interface Guide

The user interface of SellWise was built using Bootstrap 5 to ensure it is clean, intuitive, and mobile-friendly. Below is a guide to navigating the primary sections of the application.

6.3.1 Getting Started and Authentication

When you launch the system, the first page you encounter is the **User Login** screen. This page restricts access so that only authorized personnel can view sensitive store data. You must enter your registered email address and password. If you do not have an account, the **Sign Up** page allows you to register and automatically creates a new isolated store workspace for you.

6.3.2 Dashboard and Analytics

Immediately after logging in, you will be directed to the **Dashboard**. This is the central hub for monitoring your business.

- **KPI Cards:** At the top, you will see today's total revenue, the number of orders processed today, and any pending orders.
- **Sales Charts:** Interactive graphs show your sales performance over the past week and month.
- **Machine Learning Forecast:** A dedicated chart displays the Prophet-based prediction of your product demand for the next 30 days, helping you decide when to restock.

6.3.3 Point of Sale (Orders)

To process a customer's purchase, navigate to the **Orders** section.

- You can search for products using the search bar and click to add them to the current cart.

- You can adjust quantities and apply discounts or delivery charges.
- Before finalizing the sale, you can select an existing customer from the dropdown or quickly register a new customer.
- Clicking "Complete Order" will safely deduct the items from your inventory and log the sale.

6.3.4 Inventory and Alerts

To manage your stock, go to the **Products** page. Here, you can view your entire catalog, add new items, and update prices or stock quantities. If an item's stock falls below its defined minimum threshold, it will trigger an alert. You can view these critical warnings on the **Inventory Alerts** page, ensuring you never run out of popular items unexpectedly.

6.3.5 Customer Management

The **Customers** page displays a comprehensive directory of your buyers. Alongside their contact information, the system displays their automatically calculated RFM segment (such as "Champion", "Loyal", or "At Risk"). This helps you identify which customers to reward and which ones you might need to re-engage.

6.4 Conclusion

This chapter detailed the hardware and software prerequisites needed to operate SellWise successfully. It also provided a foundational walkthrough of the user interface, outlining how to perform daily operations from processing sales to analyzing AI-driven forecasts. The intuitive design ensures that shop employees require minimal training to become proficient.

Chapter 7

CONCLUSION AND FUTURE WORKS

7.1 Conclusion

The SellWise project was developed to address a significant operational gap in the retail sector: small and medium-sized shop owners desperately need modern software to manage their businesses, but existing enterprise tools are often too complicated, bloated, or expensive. Throughout the lifecycle of this project, we successfully engineered a complete, full-stack web application that handles point-of-sale operations, tracks inventory in real-time, automatically segments customers, and leverages machine learning to predict future sales.

Building this system highlighted the importance of a clean architectural design. By enforcing strict multi-tenancy at the database level and handling financial transactions safely through Entity Framework Core, we ensured that the system is both secure and reliable. Furthermore, separating the computationally heavy Python prediction code into an isolated microservice guaranteed that the main C# web application remained fast and responsive for cashiers processing live orders.

7.2 Outcomes

The successful implementation of SellWise yielded several positive outcomes:

- **Centralized Management:** Shop owners can now visualize all their critical business data on a single, unified dashboard, drastically reducing the time spent compiling paper records or managing disparate spreadsheets.
- **Proactive Restocking:** The integration of the Meta Prophet machine learning model helps owners accurately forecast upcoming product demand, preventing stockouts of popular items and reducing the financial burden of overstocking.
- **Automated Customer Insights:** By automatically calculating RFM scores, the system instantly identifies the most valuable customers without requiring the owner to perform complex data analysis manually.
- **Data Integrity:** The robust order processing pipeline ensures that inventory numbers are strictly tied to finalized sales, eliminating the discrepancies that plague manual systems.

- **Early Warnings:** Automated background tasks continuously scan the database to provide managers with early warning alerts before an item reaches critical stock levels.

7.3 Current Limitations

While the system fulfills its primary objectives exceptionally well, we have identified a few limitations during testing that should be noted:

- **Single ML Model Dependency:** The demand forecasting currently relies entirely on the Prophet library. While effective for general seasonal trends, the system does not evaluate or compare alternative machine learning models (such as ARIMA or neural networks) that might yield higher accuracy for highly volatile products.
- **Network Dependency:** SellWise is a web-based application requiring an active network connection to process sales. There is currently no localized offline mode to queue transactions if the internet or local server connection drops.
- **Single Currency Constraint:** A registered store is bound to a single type of currency and taxation rule. This could present challenges for shops operating near international borders that accept multiple currencies.

7.4 Future Works

To further evolve SellWise into an enterprise-grade retail platform, several enhancements are planned for future development iterations:

- **Automated Marketing Integrations:** We plan to integrate the RFM customer segmentation data directly with email marketing APIs (such as SendGrid or Mailchimp). This would allow the system to automatically send promotional codes or "We miss you" emails to customers categorized as "At Risk".
- **Mobile Application Counterpart:** Developing a companion Android and iOS application using frameworks like Flutter or .NET MAUI would empower managers to review dashboard analytics remotely and allow employees to process inventory checks using their smartphone cameras as barcode scanners.
- **Cloud Deployment Architecture:** Transitioning the current Docker-based local setup into a fully managed cloud architecture (using services like AWS ECS or Azure App Services) would enhance scalability, provide automated off-site backups, and drastically simplify the onboarding process for new store owners.
- **Multi-Branch Support:** Enhancing the system to support a single owner managing shared inventory across multiple geographic store branches, allowing for inter-store stock transfers.